Deloitte.

ABAP on HANA:
New Syntax and Features

Deloitte Technology Academy (DTA)

Agenda

Topic	Descriptions	Duration
ABAP on HANA: New Syntax and Features Overview	Inline Declarations—Data(), Field Symbols(), Constructor Expressions—Value(), CONV(), COND(), SWITCH(), Table Expressions—Table Line, Line Exists(), Line Index(), Filter, Reduce, Chaining, Corresponding, For…Let, Base, and Character/String Processing	4 hours
Open SQL Overview	Open SQL Introduction, Features, and Rule Changes for Join	

Copyright © 2022 Deloitte Development LLC. All rights reserved.

Learning Objectives

By the end of this session, you will be able to:

- Discuss New Syntax
- Describe Inline Declarations
- Discuss Constructor Expressions
- List key Table Expressions
- Discuss the features of Open SQL

Copyright © 2022 Deloitte Development LLC. All rights reserved.

ABAP on HANA: New Syntax and Features Overview

What is New Syntax?

- SAP has introduced an expression-oriented approach referred to as the 'New Syntax' in SAP ABAP from ABAP 7.40 onwards.
- There are various new elements such as Inline Declarations, Constructor Expressions, and Table Expressions.

Copyright © 2022 Deloitte Development LLC. All rights reserved.

Inline Declarations

- The new declaration operators DATA and FIELD-SYMBOL allow inline declarations of variables and field symbols with declaration expressions in declaration positions.
- Declaration positions are written positions where the operand type can be determined from the context statically.

DATA(var)

FIELD-SYMBOL(<fs>)

Copyright © 2022 Deloitte Development LLC. All rights reserved.

Inline Declarations—Data()

Examples of declaration positions for inline declaration DATA(var)

```
LOOP AT itab INTO DATA(wa).
...
ENDLOOP.
READ TABLE itab INTO DATA(wa) ...
```

```
oref->meth( IMPORTING p1 = DATA(a1)
IMPORTING p2 = DATA(a2)... ).
```

```
FIND ... IN ... MATCH COUNT
DATA(cnt).
```

```
DATA(ref) = class=>factory( ... ).
```

```
CALL TRANSFORMATION ... RESULT XML DATA(xml).
```


Copyright © 2022 Deloitte Development LLC. All rights reserved.

Inline Declarations—Data() (Cont.)

Example for usage

```
DATA ixml                   TYPE REF TO if_ixml.
DATA stream_factory         TYPE REF TO if_ixml_stream_factory.
DATA document               TYPE REF TO if_ixml_document.

ixml               = cl_ixml=>create().
stream_factory     = ixml->create_stream_factory().
document           = ixml->create_document().
```

```
DATA (ixml)                 = cl_ixml=>create().
DATA (stream_factory)       = ixml->create_stream_factory().
DATA (document)             = ixml->create_document().
```


Copyright © 2022 Deloitte Development LLC. All rights reserved.

Inline Declarations—Data() (Cont.)

Example for usage

```
SELECT so_id,
               currency_code,
               gross_amount,
               delivery_status
FROM snwd_so
INTO TABLE @DATA(lt_result).
```


Copyright © 2022 Deloitte Development LLC. All rights reserved.

Inline Declarations—Field Symbols()

Declaration positions for inline declaration FIELD-SYMBOL(<fs>)

```
ASSIGN ... TO FIELD-SYMBOL(<fs>).
```

```
LOOP AT itab ASSIGNING FIELD-SYMBOL(<line>).
...
ENDLOOP.
```

```
READ TABLE itab ASSIGNING FIELD-SYMBOL(<line>).
```


Copyright © 2022 Deloitte Development LLC. All rights reserved.

Inline Declarations—Field Symbols() (Cont.)

Example for usage

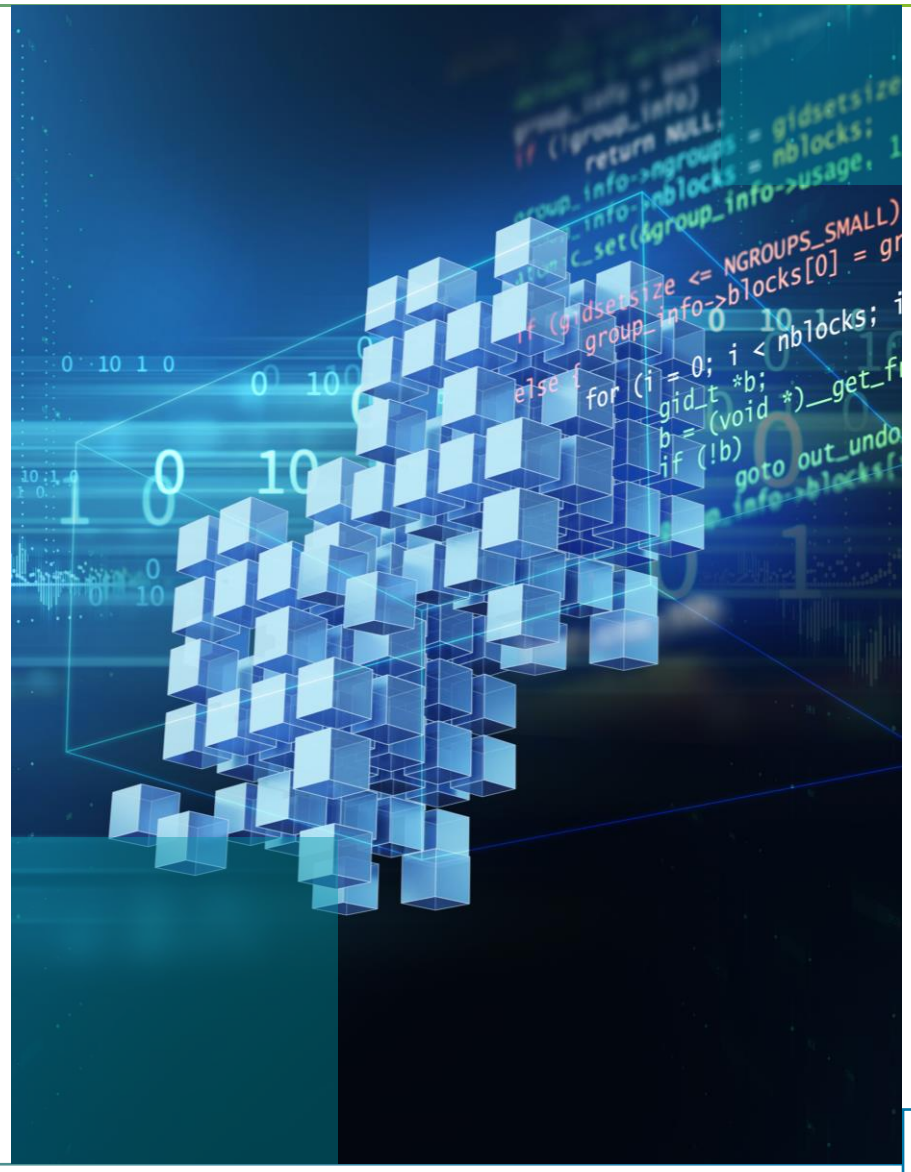
```
FIELD-SYMBOLS <line> LIKE LINE OF itab.
LOOP AT itab ASSIGNING <line>.
...
ENDLOOP.
```

```
LOOP AT itab ASSIGNING FIELD-SYMBOL(<line>).
...
ENDLOOP.
```


Copyright © 2022 Deloitte Development LLC. All rights reserved.

Constructor Expressions

- The new constructor operators allow constructing results of a specified type in general expression positions
  - **VALUE** creates values
  - **CONV** converts values
  - **COND** and **SWITCH** enable conditional expressions



Copyright © 2022 Deloitte Development LLC. All rights reserved.

Constructor Expressions—Value()

Value operator VALUE

```
DATA itab TYPE t_itab
DATA wa  LIKE LINE OF itab.
Wa-col1 = 1. wa-col2 = 2.
APPEND wa TO itab.
Wa-col1 = 3. wa-col2 = 4.
APPEND wa TO itab.
Meth( itab ).
```

```
DATA(itab) = VALUE t_itab(
  ( col1 = col2 = 2 )
  ( col1 = col2 = 4 ) ).
```

```
Meth( VALUE t_itab(
  ( col1 = 1 col2 = 2 )
  ( col1 = 3 col2 = 4 ) ) ).
```

Copyright © 2022 Deloitte Development LLC. All rights reserved.

Constructor Expressions—CONV()

Conversion operator CONV

```
DATA text TYPE c LENGTH 255.
DATA helper TYPE string.
helper = text.
DATA(xstr) = cl abap codepage=>convert to
                    ( source = helper).
```

```
DATA text TYPE c LENGTH 255.
DATA(xstr) = cl_abap_codepage=>convert_to(
                 source = CONV # ( text ) ).
```

```
IF 1 / 3 > 0.
  ...
ENDIF.
```

```
IF CONV decfloat34( 1 / 3 ) > 0.
  ...
ENDIF.
```

```
IF ‘ ‘ = ' '.
  ...
ENDIF.
```

```
IF ‘ ‘ = CONV char1( ' ' ).
  ...
ENDIF.
```


Copyright © 2022 Deloitte Development LLC. All rights reserved.

Constructor Expressions—COND()

Conditional operator COND

```
DATA time TYPE string.
IF sy-timlo < ‘120000’.
  time = | { sy-timlo TIME = ISO } AM|.
ELSEIF sy-timlo > ‘120000’.
  time = | { CONV t( sy-timlo – 12 * 3600 )
               TIME = ISP } PM|.
ELSEIF sy-timlo = ‘120000’.
  time = |High Noon|.
ELSE.
  RAISE EXCEPTION TYPE cx_cant_be.
ENDIF.
```

```
DATA(time) =
  COND string(
    WHEN sy-timlo < ‘120000’ THEN
      | { sy-timlo TIME = ISO } AM|
    WHEN sy-timlo > ‘120000’ THEN
      | { CONV t( sy-timlo – 12  * 3600 )
    WHEN sy-timlo = ‘120000’ THEN
      |High Noon|
    ELSE
      THROW cx_cant_be() ).
```


Copyright © 2022 Deloitte Development LLC. All rights reserved.

Constructor Expressions—SWITCH()

Conditional operator SWITCH

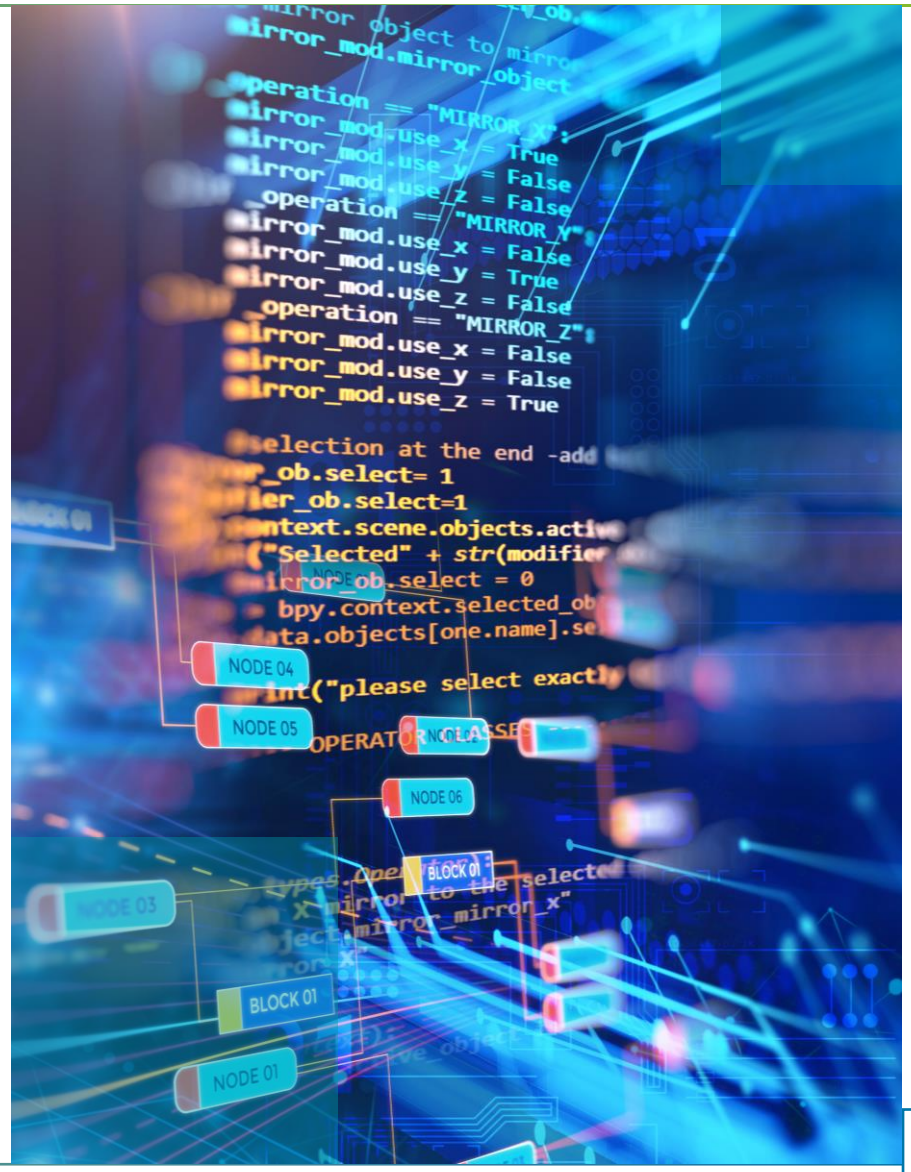
```
DATA number TYPE string.
CASE sy-index.
  WHEN 1.
    number = ‘one’.
  WHEN 2.
    number = ‘two’.
  WHEN 3.
    number = ‘three’.
  WHEN OTHERS.
    RAISE EXCEPTION TYPE cx_overflow.
ENDCASE.
```

```
DATA(number) =
  SWITCH string( sy-index
                 WHEN 1 THEN ‘one’
                 WHEN 2 THEN ‘two’
                 WHEN 3 THEN ‘three’
                 ELSE THROW cx_overflow() ).
```


Copyright © 2022 Deloitte Development LLC. All rights reserved.

Table Expressions

- The new table expressions `itab[ … ]` enable read access to internal tables at operand positions.
- The operand positions can be read positions and also some write positions
- Table expressions are LHS expression



Copyright © 2022 Deloitte Development LLC. All rights reserved.

Table Expressions—Table Line

Line Specification

```
READ TABLE itab INDEX idx INTO wa.
```

```
wa = itab[ idx ].
```

```
READ TABLE itab INDEX idx
  USING KEY key INTO wa.
```

```
wa = itab[ KEY key INDEX idx ].
```

```
READ TABLE flights WITH KEY
  Col1 = ... Col2 = ... INTO wa.
```

```
wa = itab[col1 = ... Col2 = ... ].
```

```
READ TABLE flights WITH TABLE KEY key
  COMPONENTS col1 = ... Col2 = ...
  INTO wa.
```

```
Wa = itab[ KEY key COMPONENTS
           col1 = ... Col2 = ...].
```

```
Wa = itab[ KEY key
           col1 = ... Col2 = ...].
```


Copyright © 2022 Deloitte Development LLC. All rights reserved.

Table Expressions—Line Exists(), Line Index()

< / >

Line Exists()

- This is an alternate for Read Table TRANSPORTING NO FIELDS
- Does not return Index value

Line Index()

- This will return the sy-tabix value of the row of the table where the value resides

Copyright © 2022 Deloitte Development LLC. All rights reserved.

Table Expressions—Filter, Reduce

< / >

Filter

- In this variant, you simply extract the lines from an internal table into a tabular result, that fulfills a simple value condition

Reduce

- Constructor operator REDUCE for constructing a value using iteration expressions with FOR

Copyright © 2022 Deloitte Development LLC. All rights reserved.

Table Expressions—Chaining, Corresponding

</>

Chaining

- The following chainings with table expressions are possible:
- … `itab[ …]-comp`
- … `struct-comp[ … ] …`
- … `itab[ … ][ … ] …`

Corresponding

- The Corresponding operator can be used to transfer content from one table to another by transferring the corresponding components

Copyright © 2022 Deloitte Development LLC. All rights reserved.

Table Expressions—For … Let, Base

< / >

For… Let

- FOR is the Iteration Expression. An iteration expression is to perform the iteration on the table.
- The FOR can be used with Constructor Operators – like NEW, and VALUE. It is like the LOOP but using a different pattern.

Base

- It makes the RHS known to the LHS.
- In simple words, it is used as an alternative to append statement.

Copyright © 2022 Deloitte Development LLC. All rights reserved.

New Built-In Function: ipow

Example:

```
Cl_demo_output=>display( |** : { ‘1.2’ ** 2 } \n| &&
                         |ipow: { ipow( base = ‘1.2’ exp = 2 ) }| ).
```

Output:

```
** : 1.4399999999999999
ipow: 1.4400
```

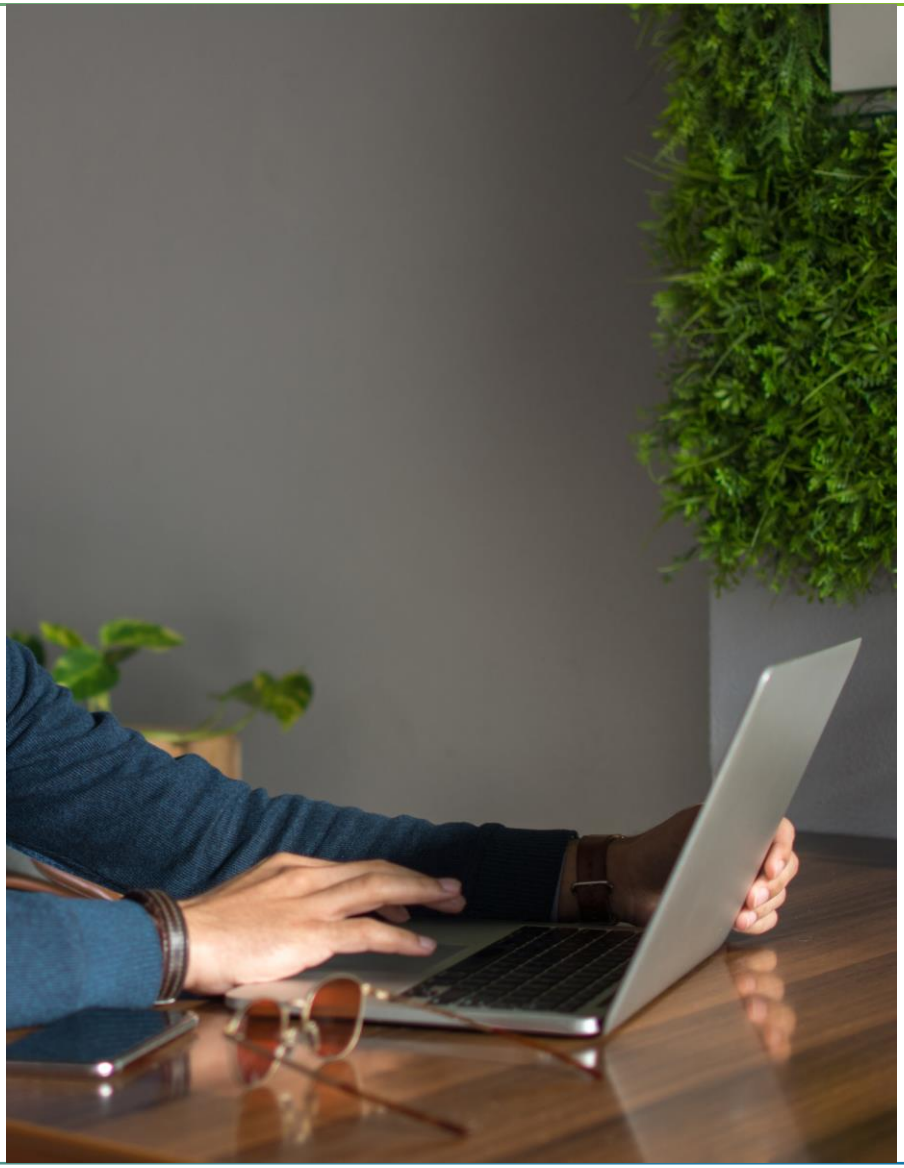

Copyright © 2022 Deloitte Development LLC. All rights reserved.

New Formatting Options for Character/String Processing

Alpha	Helps in using conversion exit ALPHA without using the FM
Concatenation	‘&&’ operator concatenates two data variables
String Templates (Piping)	‘	‘ operator can be used in place of ‘ ‘ for strings
To_upper and to_lower	Converts to alternate cases
Between	Exceptions can be handled within the conversion statement

Copyright © 2022 Deloitte Development LLC. All rights reserved.

Open SQL Overview



Open SQL

Open SQL defines the syntax and semantics used for accessing all the relational database systems supported by SAP. It allows ABAP developers to write database-independent code. It is considered the first abstraction layer in the ABAP platform.

Open SQL syntax and features:

- Comma separated
- Escape characters for host variables
- SQL expressions can be done, such as:
  - Arithmetic
  - String concatenation

Copyright © 2022 Deloitte Development LLC. All rights reserved.

Where is Open SQL Useful?

</>

- Normal Select
- Aggregation
- Open SQL literal
- To check generic existence
- Arithmetic expressions
- Simple case
- Searched Case
- Searched case (advanced)
- Coalesce
- Group by

Copyright © 2022 Deloitte Development LLC. All rights reserved.

Open SQL Rule Changes for Join

- It is no longer the case that all comparisons of the ON condition must contain a column from a database table or view on the right side as an operand
- The right side of a join expression is no longer restricted to single tables or views
- The right side can itself be a (parenthesized) join expression whose results set is then evaluated
- LEFT OUTER JOIN and RIGHT OUTER JOIN can be used
- Fields from the right side in the WHERE condition of the current SELECT statement can be specified in LEFT OUTER JOIN
- Fields from the left side can be specified in RIGHT OUTER JOIN

Copyright © 2022 Deloitte Development LLC. All rights reserved.

Resources

- http://zevolving.com/category/concepts/
- http://zevolving.com/2015/05/abap-740-for-iteration-expression/
- http://zevolving.com/2014/09/abap-740-value-operator-create-itab-entries/
- https://blogs.sap.com/2015/10/25/abap-740-quick-reference/
- https://blogs.sap.com/2016/03/02/old-and-new-abap-syntax-overview-sheet/

Note: In your SAP system you can go to -> ABAP examples -> ABAP Release specific changes

Copyright © 2022 Deloitte Development LLC. All rights reserved.

Summary

Here are the key learning points of the module.

- The new declaration operators DATA and FIELD-SYMBOL allow inline declarations of variables and field symbols with declaration expressions in declaration positions.
- The new constructor operators allow constructing results of a specified type in general expression positions
- The new table expressions itab[…] enable read access to internal tables at operand positions.
- The right side of a join expression is no longer restricted to single tables or views

Copyright © 2022 Deloitte Development LLC. All rights reserved.

Thank You

Deloitte.

About Deloitte
Deloitte refers to one or more of Deloitte Touche Tohmatsu Limited, a UK private company limited by guarantee ("DTTL"), its network of member firms, and their related entities. DTTL and each of its member firms are legally separate and independent entities. DTTL (also referred to as "Deloitte Global") does not provide services to clients. In the United States, Deloitte refers to one or more of the US member firms of DTTL, their related entities that operate using the "Deloitte" name in the United States and their respective affiliates. Certain services may not be available to attest clients under the rules and regulations of public accounting. Please see www.deloitte.com/about to learn more about our global network of member firms.

This communication contains general information only, and none of Deloitte Touche Tohmatsu Limited ("DTTL"), its global network of member firms or their related entities (collectively, the "Deloitte organization") is, by means of this communication, rendering professional advice or services. Before making any decision or taking any action that may affect your finances or your business, you should consult a qualified professional adviser.

No representations, warranties or undertakings (express or implied) are given as to the accuracy or completeness of the information in this communication, and none of DTTL, its member firms, related entities, employees or agents shall be liable or responsible for any loss or damage whatsoever arising directly or indirectly in connection with any person relying on this communication. DTTL and each of its member firms, and their related entities, are legally separate and independent entities.

Copyright © 2022 Deloitte Development LLC. All rights reserved.